

Výstupy ŠVP

INF-08e Vytvořím přehledný program pro vyřešení konkrétního problému s ohledem na jeho možné důsledky a svou odpovědnost za ně. (*Informatika 4, 2/2 · z INF-08*)

INF-06 Analyzuji problém, rozdělím problém na menší části, rozhodnu, které je vhodné řešit algoritmicky, své rozhodnutí zdůvodním; sestavím a zapíšu algoritmy pro řešení problému. (*Informatika 4, 2/2*)

INF-09 Ověřím správnost, najdu a opravím případnou chybu v algoritmu, otestuji, odladím a optimalizuji program. (*Informatika 4, 2/2*)

Učivo: vývoj programu – volba nástroje, návrh uživatelského rozhraní, nápověda a dokumentace, autorství a licence, etika programátora

Lehké úlohy procvičí čitelnost, licence, verze a etiku; střední se týkají návrhu funkcí, rozhraní a plánování projektu; těžké řeší návrh datových struktur, testovatelnost, přístupnost a odpovědnost programátora. Pojmy a vzorce máš v listu Must know. Vyber si, kolik úloh spočítáš; řešení si ukládej do svého portfolia.

Lehké ★☆☆

1. Které názvy proměnných jsou výstižné a které ne? x1, pocet_zaku, data2, cena_s_dph, f, prumerna_teploata, a, seznam. Navrhni lepší názvy pro ty nevýstižné (představ si program na evidenci výsledků běhu).
2. Co znamenají licence Creative Commons: CC BY, CC BY-SA, CC BY-NC a CC0? Kterou smíš použít pro obrázek v aplikaci, kterou budeš prodávat?
3. Program chce délku trasy v km. Seřaď hlášky při chybném vstupu od nejhorší po nejlepší a zdůvodni: a) „Chyba“; b) „ValueError: could not convert string to float“; c) „Zadej délku trasy v km jako kladné číslo, např. 125 nebo 12.5.“; d) „Špatný vstup, zkus to znovu.“
4. Program má verzi 2.3.1 (hlavní.vedlejší.oprava). Jaké číslo dostane další verze, když a) opraviš chybu, b) přidáš novou funkci a starší věci fungují dál, c) změníš formát ukládaných dat tak, že staré soubory nepůjdou načíst?
5. Napiš specifikaci programu „kalkulačka spropitného“: vstupy (typ, rozsah, jednotky), výstupy a aspoň dvě podmínky řešení. Co program dělat nebude?
6. Zadavatel chce do tvé aplikace: a) tmavý režim; b) tajně odesílat kontakty z telefonu na server; c) sbírat polohu uživatelů bez jejich vědomí „pro statistiky“; d) formulář pro zpětnou vazbu. Které požadavky bys odmítl a proč?

Střední ★★☆☆

7. Program třikrát opakuje výpočet plochy místnosti (4 × 3,5 m; 3 × 3 m; 5 × 4,2 m) a výsledky sčítá. Přepiš ho pomocí funkce obsah(delka, sirka) a cyklu přes seznam dvojic. Kolik je celková plocha? Co získáš, když přibude čtvrtá místnost?
8. Napiš funkci spropitne(castka, procento=10) s docstringem, která vrátí spropitné zaokrouhlené nahoru na celé koruny (math.ceil). Otestuj: útrata 487 Kč s výchozím procentem, 1 200 Kč s 15 % a 99 Kč s 0 %.
9. Co program vypíše, když uživatel postupně zadá 2, 5, 1 a 0? Proč je u menu vhodný cyklus while True s break?

```
while True:
    volba = input()
    if volba == "1":
        print("Nová hra")
    elif volba == "2":
        print("Nejlepší skóre: 120")
    elif volba == "0":
        print("Konec")
        break
    else:
        print("Neznámá volba, zadej 0, 1 nebo 2")
```

10. Projekty se hodnotí body 0–10 s vahami: funkčnost 40 %, čitelnost kódu 20 %, dokumentace 20 %, prezentace 20 %. Projekt A má body 8, 6, 5, 9, projekt B 7, 9, 8, 6. Který vyhraje? Co z výsledku plyne pro tvou práci na projektu?

11. Porovnej dvě verze funkce a vypiš, které řádky se změnily (jako příkaz diff: „-“ odebraný, „+“ přidaný řádek). Popiš změnu jednou větou jako zprávu k uložení verze (commit).

```
# verze 1
def cena(km, spotreba, litr):
    litry = km * spotreba / 100
    return litry * litr

# verze 2
def cena(km, spotreba, litr, osoby=1):
    litry = km * spotreba / 100
    return round(litry * litr / osoby, 2)
```

12. Plán projektu: návrh 2 h, programování 6 h, testování 3 h, dokumentace 2 h. Zkušenost říká, že začínající programátoři podceňují čas asi o 50 %. Kolik hodin si máš naplánovat? Kolik týdnů to je při 3 hodinách práce týdně? Kterou činnost studenti nejčastěji vynechají?

Těžké ★★ ★

13. Navrhni evidenci výpůjček knihovny: výpůjčka je slovník (kniha, čtenář, den výpůjčky, vráceno) v seznamu. Co program vypíše? Proč je výhodné rozdělit evidenci na funkce pujcit, vratit a zpozdene? Jaký test bys přidal?

```
vypujcky = []

def pujcit(kniha, ctenar, den):
    vypujcky.append({"kniha": kniha, "ctenar": ctenar, "den": den, "vraceno": False})

def vratit(kniha):
    for v in vypujcky:
        if v["kniha"] == kniha and not v["vraceno"]:
            v["vraceno"] = True
            return True
    return False

def zpozdene(dnes, lhuta=30):
    return [v["kniha"] for v in vypujcky if not v["vraceno"] and dnes - v["den"] > lhuta]

pujcit("Malý princ", "Eva", 1)
pujcit("Hobit", "Jan", 10)
pujcit("Babička", "Eva", 25)
print(vratit("Hobit"), vratit("Duna"))
print(zpozdene(45))
print(zpozdene(60))
```

14. Svůj projekt chceš zveřejnit pod licenci MIT a použít v něm knihovnu s licenci GPL. Co to znamená, když program budeš šířit? Jaké máš možnosti?
15. Přístupnost: kontrast textu se počítá jako $\frac{L_1 + 0,05}{L_2 + 0,05}$, kde $L_1 \geq L_2$ jsou relativní jasy barev (bílá $L = 1$, černá $L = 0$, žlutá $L \doteq 0,93$, tmavě modrá $L \doteq 0,02$, šedá #777777 $L \doteq 0,18$). Pro běžný text je doporučeno aspoň 4,5 : 1. Spočítej kontrast bílého textu na žluté, černého na bílé, tmavě modrého na bílé a šedého na bílé.
16. Firma chce program, který vybírá uchazeče o práci; naučí ho na datech z minulých let, kdy firma přijala 60 % mužů a 30 % žen se stejnou kvalifikací. Co bude program dělat? Proč je to problém i přesto, že „jen počítá z dat“? Co může programátor udělat?

17. Původní program míchal výpočet s input a print, takže nešel automaticky testovat. Přepsaná funkce `cena_taxi` vrací cenu (nástupní 40 Kč + 28 Kč/km, v noci +20 %). Projdou všechny testy? Doplň test hraniční hodnoty a vysvětli, proč je oddělení výpočtu od vstupu a výstupu výhodné.

```
def cena_taxi(km, noc=False):
    cena = 40 + 28 * km
    if noc:
        cena = cena * 1.2
    return round(cena, 2)

assert cena_taxi(0) == 40
assert cena_taxi(5) == 180
assert cena_taxi(5, noc=True) == 216
assert cena_taxi(2.5) == 110
print("testy prošly")
```

18. Aplikace pro každý z 10 000 objednaných produktů hledá, zda je ve skladu (seznam 10 000 položek), lineárním vyhledáváním. Kolik porovnání to nejhůř znamená? Jak se to změní, když sklad uložíš do množiny set (vyhledání je přibližně jeden krok)? Proč se na efektivitu musí myslet už při návrhu projektu (→ H60)?

1. výstižné: pocet_zaku, cena_s_dph, prumerna_teploata; nevýstižné: x1, data2, f, a, seznam (neříkají, co obsahují) – lépe např. cas_behu, vysledky_2026, nejlepsi_cas, pocet_bezcu, casy_zavodniku
2. CC BY – použít i upravit, uvést autora; CC BY-SA – totéž a úpravy pod stejnou licenci; CC BY-NC – jen nekomerčně; CC0 – volné dílo bez podmínek; do prodávané aplikace CC BY, CC BY-SA (s dodržáním podmínek) nebo CC0, ne NC
3. nejhorší b) (technická hláška, uživatel netuší, co dělat), a), d) (ví, že je chyba, ale ne jaká), nejlepší c) – řekne co, v jakých jednotkách a v jakém tvaru zadat (s příkladem)
4. a) 2.3.2; b) 2.4.0; c) 3.0.0 (nekompatibilní změna zvyšuje hlavní číslo)
5. vstupy: útrata (desetinné číslo > 0 , Kč), procento spropitného (celé číslo 0–30, výchozí 10), případně počet osob (celé ≥ 1); výstupy: spropitné a celková částka (na osobu), zaokrouhleno na koruny; podmínky: kladná útrata, počet osob ≥ 1 ; nebude např. převádět měny
6. odmítnout b) a c) – sběr osobních údajů bez vědomí a souhlasu uživatele je neetický a v EU nezákonný (GDPR); a) a d) jsou v pořádku
7. def obsah(delka, sirka): return delka * sirka; for d, s in místnosti: celkem += obsah(d, s) $\rightarrow 44.0 \text{ m}^2$ (14 + 9 + 21); novou místnost jen připišeš do seznamu, výpočet se nemění
8. return math.ceil(castka * procento / 100); docstring popíše parametry (Kč, procenta) a návratovou hodnotu; výsledky 49 Kč (celkem 536 Kč), 180 Kč, 0 Kč
9. Nejlepší skóre: 120 / Neznámá volba, zadej 0, 1 nebo 2 / Nová hra / Konec; menu se má opakovat, dokud uživatel nezvolí konec – počet opakování předem neznáme; neplatná volba program neshodí, jen vypíše nápovědu
10. A: $0,4 \cdot 8 + 0,2 \cdot (6 + 5 + 9) = 7,2$; B: $0,4 \cdot 7 + 0,2 \cdot (9 + 8 + 6) = 7,4 \rightarrow$ vyhraje B; i funkční program ztrácí body, když je nečitelný a bez dokumentace
11. změnil se 1. a 3. řádek: – def cena(km, spotreba, litr): + def cena(km, spotreba, litr, osoby=1): – return litry * litr + return round(litry * litr / osoby, 2); zpráva např. „Cena cesty se dělí mezi osoby a zaokrouhluje na haléře“
12. plán 13 h, s rezervou $13 \cdot 1,5 = 19,5$ h; při 3 h týdně 7 týdnů; nejčastěji se šidí testování a dokumentace – nechat na ně čas hned v plánu
13. True False / ['Malý princ'] / ['Malý princ', 'Babička']; každá funkce dělá jednu věc a jde otestovat zvlášť; testy např. vrácení už vrácené knihy (False), výpůjčka přesně 30 dní (ještě není zpožděná), prázdná evidence
14. GPL je „copyleft“: kdo šíří program, který ji obsahuje, musí celé dílo zveřejnit také pod GPL (se zdrojovým kódem); možnosti: dát celý projekt pod GPL, najít knihovnu s volnější licencí (MIT, BSD), nebo funkci napsat sám
15. bílá/žlutá $\approx 1,07:1$ – nečitelné; černá/bílá 21:1; tmavě modrá/bílá $\approx 15,3:1$ – výborné; šedá/bílá $\approx 4,48:1$ – těsně pod doporučenou hranici
16. naučí se, že být muž „zvyšuje šanci“, a bude ženy znevýhodňovat – zopakuje dřívější nespravedlnost (zkreslená data $\rightarrow$ zkreslený model); programátor má upozornit zadavatele, testovat výsledky modelu podle skupin, nepoužívat pohlaví ani údaje, které ho prozrazují, a nechat konečné rozhodnutí na člověku
17. program vypíše testy prošly (40; 180; 216; 110); doplnit např. záporné km (funkce by měla vyvolat chybu) nebo velmi dlouhou jízdu; výpočet ve funkci lze otestovat bez uživatele a použít i v jiném rozhraní (web, mobil)
18. $10\,000 \cdot 10\,000 = 10^8$ porovnání (sekundy až minuty); s množinou asi 10^4 kroků – tisíce krát rychleji; se zvětšením dat roste lineární hledání kvadraticky, a co funguje na testovacích datech, může u skutečných uživatelů selhat